

ReLibra: Routing-Replay-Guided Load Balancing for MoE Training in Reinforcement Learning

Chao Jin* Xinming Wei* Yinmin Zhong* Chengxu Yang† Bingyang Wu*
 Ruidong Zhu* Zili Zhang* Yuliang Liu† Xin Jin*

*School of Computer Science, Peking University †Independent Researcher

Abstract

Load imbalance is a long-standing challenge in Mixture-of-Experts (MoE) training and is exacerbated in reinforcement learning (RL) for LLMs, where hot experts can shift frequently across micro-batches. Existing MoE training systems rely on historical loads to predict future expert demand, making them less effective under sharp fluctuations.

We propose ReLibra, an MoE RL training system that exploits a unique opportunity in RL’s rollout-training workflow, **routing replay**, to enable **fine-grained load balancing at micro-batch granularity**. Because rollout and training process the same tokens with the same MoE parameters, the token-to-expert routing decisions are known before training starts. Leveraging this information, ReLibra places two MoE load-balancing mechanisms at inter- and intra-batch timescales, matching their communication patterns to hierarchical network bandwidths. At the inter-batch timescale, ReLibra performs **expert reordering** to redistribute experts for batch-level cross-node balancing; at the intra-batch timescale, it dynamically performs **expert replication** within a node to absorb micro-batch-level load fluctuations. Experiments on diverse MoE LLMs and RL workloads show that ReLibra improves training throughput by up to $1.6\times$ over Megatron-LM and by up to $1.2\times$ over EPLB, even when EPLB is given oracle loads. Moreover, ReLibra remains within 6%–10% of the throughput of an idealized balanced baseline.

1 Introduction

Mixture-of-Experts (MoE) [1–3] has emerged as a mainstream architecture for scaling large language models (LLMs) [4–9], as it increases model capacity without proportionally increasing computation by sparsely routing tokens to a small set of experts [10, 11]. However, MoE training in both pretraining and RL post-training [12, 13] suffers from load imbalance [14–18], because dynamic routing can overload GPUs hosting hot experts under expert parallelism (EP) [6, 11, 19], creating stragglers and reducing training efficiency.

While MoE pretraining typically exhibits a relatively stable imbalance pattern, MoE RL training shows a more fluctuating one (§ 2.2). We use the term imbalance pattern to describe how expert load distribution varies across micro-batches, where a training batch is split into multiple micro-batches that are processed separately to reduce activation memory. In RL training, the set of hot experts can change substantially from one micro-batch to the next, whereas in pretraining it

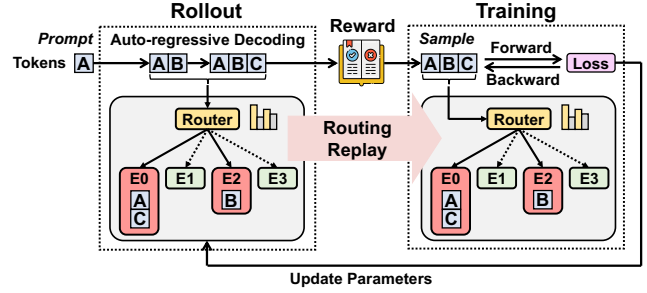


Figure 1: MoE RL training workflow and the opportunity for routing replay.

remains relatively stable. This difference arises from two main reasons. First, while MoE pretraining is conducted on web-scale, diverse corpora, MoE RL training typically targets narrower domains [8, 13, 20] (e.g., math [21], coding [22], and instruction following [23]). As a result, micro-batches from different domains in RL training tend to activate different sets of hot experts. Second, MoE pretraining employs algorithmic load-balancing mechanisms to prevent some experts from being undertrained, such as auxiliary balancing losses [1–3] and routing bias [6], whereas these mechanisms are removed during RL training [9, 21, 24], since experts have already been shaped during pretraining. Without such mechanisms, expert load distributions in RL training become more skewed and unstable, even within a single domain.

Existing MoE training systems struggle to cope with such fluctuating imbalance patterns because they are primarily designed for pretraining, where expert load distributions are relatively stable and therefore predictable. For example, EPLB [25] periodically replicates hot experts based on recent load statistics, typically once every one or several training batches to amortize the reconfiguration overhead. PopFetcher [17] prefetches experts using activation patterns from recent training batches in adjacent layers. Despite their different mechanisms, both systems rely on the same assumption: future expert demand can be predicted from recent history. When expert load distributions fluctuate sharply at micro-batch granularity in RL training, this assumption breaks down, limiting the effectiveness of prior solutions.

Although RL makes expert load distributions more unstable, its workflow also creates a unique opportunity for fine-grained load balancing via **routing replay**. As shown in Figure 1, RL consists of a rollout stage that generates responses

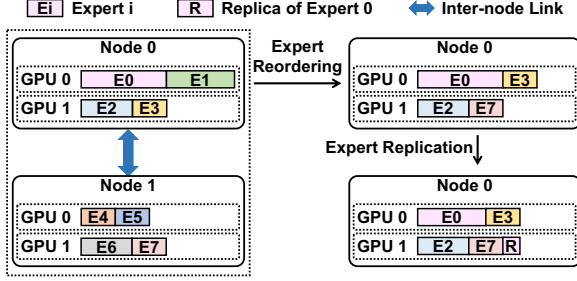


Figure 2: Load-balancing mechanisms for MoE training.

and a training stage that updates the model on the generated samples. Since both stages process the same token sequence with the same MoE parameters, they produce the same routing scores and, theoretically, the same top- k experts. Thus, the exact token-to-expert routing decisions are determined during rollout and can be replayed in training. Moreover, while some RL algorithms [26, 27] perform multiple updates per rollout, recent industrial practice [20, 28, 29] has also adopted routing replay so that experts are updated with the exact tokens they generated, which may improve algorithmic training stability.

However, even with routing known in advance, finding the optimal load-balancing plan for each micro-batch and MoE layer remains difficult. As shown in Figure 2, MoE training systems provide two main mechanisms: expert reordering, which redistributes experts across the EP group, and expert replication, which adds replicas for overloaded experts. Applying them at micro-batch granularity results in a large combinatorial optimization problem, and even simplified variants are NP-hard [30], making exact solving impractical.

To this end, we propose ReLibra, an MoE RL training system that exploits the routing replay opportunity in RL to address fluctuating load imbalance at micro-batch granularity. The key insight of ReLibra is to place the two system-level load-balancing mechanisms at inter- and intra-batch timescales by matching their communication patterns with the hierarchical network bandwidths in training clusters.

To scale MoE models with massive numbers of experts, cross-node expert parallelism is widely adopted [6, 8, 9, 19]. However, because imbalance patterns vary rapidly across micro-batches, frequent expert reordering may incur expensive communication over low-bandwidth inter-node links. By contrast, expert replication can often be restricted within a node, making it more efficient to adjust at micro-batch granularity. This asymmetry motivates a hierarchical design: ReLibra applies inter-batch (per training batch) expert reordering for coarse-grained cross-node balancing and intra-batch (per micro-batch) expert replication for fine-grained local adaptation. This decomposition makes the original problem tractable, which better matches the network topology and achieves near-optimal performance in practice across diverse MoE RL workloads, as shown in our evaluation (§ 7.1).

At the inter-batch timescale, ReLibra jointly optimizes computation and communication imbalance using routing infor-

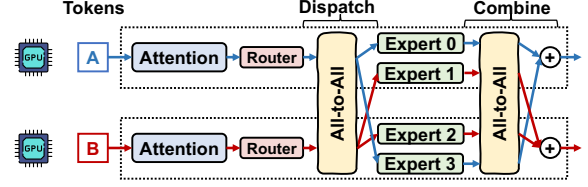


Figure 3: MoE layer and expert parallelism.

mation aggregated over the training batch. It employs a swap-based simulated annealing algorithm to efficiently search for an expert reordering plan that minimizes MoE execution time across the EP group, followed by a second round of optimization to improve data locality.

At the intra-batch timescale, ReLibra dynamically replicates experts within a node to absorb micro-batch-level load fluctuations. Its key design is a layer-shared replica buffer that stores replicas for only one layer at a time and is reused across all layers, incurring negligible memory overhead. Replica synchronization is overlapped with attention computation to avoid affecting the critical path. On top of this design, ReLibra optimizes replica placement and token splitting across replicas via a mixed-integer linear program (MILP).

In summary, we make the following contributions.

- We propose ReLibra that exploits the routing replay opportunity in MoE RL training to address fluctuating load imbalance at micro-batch granularity.
- We place expert reordering and replication at inter- and intra-batch timescales, respectively, to match their communication patterns to the network hierarchy of training clusters, thereby decomposing the original problem into two tractable sub-problems. Built on efficient algorithms and lightweight system designs, ReLibra achieves fine-grained load balancing with negligible overhead.
- We implement ReLibra and evaluate it on MoE LLMs of varying scales across diverse RL domains. Experimental results show that ReLibra improves training throughput by up to $1.6\times$ over Megatron-LM, a state-of-the-art MoE training system, and by up to $1.2\times$ over EPLB, even when EPLB is provided with oracle loads. Moreover, ReLibra attains 90%–94% of the throughput of an idealized balanced baseline.

2 Background and Motivation

2.1 Background

MoE training. MoE extends the Transformer [31] architecture by replacing the dense feed-forward network (FFN) with a sparse MoE layer containing multiple FFN experts and a trainable router. For each input token, the router selects a small set of experts (i.e., top- k experts), and the outputs of the selected experts are combined to produce the final result. This sparse activation pattern substantially increases model capacity without proportionally increasing per-token computation, making MoE a practical approach for scaling LLMs [5–11].

Domain	Dataset
Reasoning	DAPO-Math-17k [21], GPQA [36]
Instruction Following	IFBench [23]
Coding	CodeForces [22]
Mixed	Mixture of the above three domains as suggested by DeepSeek-V3.2 [20]

Table 1: Domains and datasets used in our experiments.

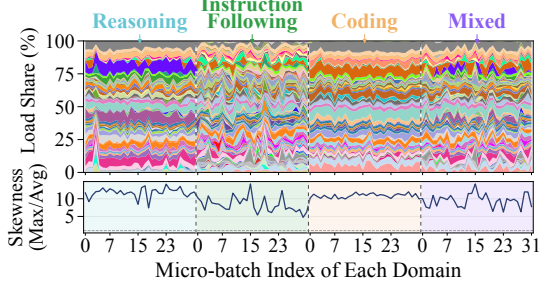


Figure 4: Dynamics of expert loads for one MoE layer across micro-batches from different domains.

The large parameter count of MoE models introduces significant memory pressure during training, making distributed training necessary. Expert parallelism (EP), tailored for MoE models, distributes different experts of an MoE layer across GPUs and relies on all-to-all communication to dispatch tokens and combine their outputs as shown in Figure 3. Recent MoE models continue to scale parameter count by increasing the number of experts. For example, Qwen3-235B-A22B [7] uses 128 experts, while DeepSeek-V3-671B [6] uses 256 experts. Consequently, cross-node EP is increasingly common in practice. Existing MoE training frameworks also combine EP with conventional parallelism strategies, such as data parallelism (DP) [32], tensor parallelism (TP) [33], and pipeline parallelism (PP) [34, 35], to jointly meet the scalability and memory requirements of large-scale training.

RL training workflow. RL for LLMs has become a key post-training technique for enhancing domain-specific capabilities by optimizing the model with external feedback on its self-generated responses, such as human feedback [12], rule-based verifiable rewards [13, 24], and generative reward models [8, 20]. Such capabilities include alignment with human preferences [12] and improved reasoning ability [13].

Each RL iteration can be viewed as a two-stage loop, as shown in Figure 1: rollout and training. In the rollout stage, the model takes a set of prompts and generates one or more responses per prompt via auto-regressive decoding [37, 38]. Post-rollout processing scores each response with a reward using either rule-based functions or a learned reward model. Some RL algorithms, such as PPO [39], also use a critic model to estimate fine-grained token-level rewards. Others like GRPO [24] use approximation strategies to reduce computational costs. A forward pass of a reference model may also be performed to compute the Kullback–Leibler (KL) divergence from the model’s initial state to stabilize training. The prompts and responses are then concatenated into train-

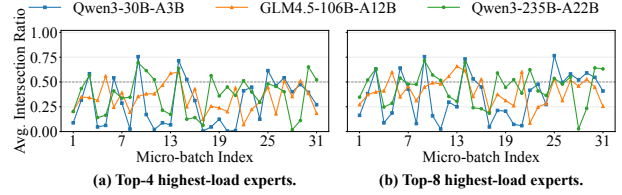


Figure 5: Average intersection ratio of top- k highest-load experts between adjacent micro-batches. Lower values indicate more diverse hot experts and therefore more fluctuating load imbalance.

ing samples, which, together with their reward signals, form a training batch. In the training stage, this batch is divided into micro-batches, each of which performs forward and backward passes and accumulates gradients until the full batch is processed. The accumulated gradients are then used to update the model parameters, which are synchronized back to the rollout workers for the next iteration.

While recent work targets the rollout stage [40, 41], our work focuses on a complementary bottleneck: improving the efficiency of the training stage itself, specifically the fluctuating load imbalance problem for MoE models.

2.2 Load Imbalance Characterization

We characterize the imbalance patterns in MoE RL training using three representative workloads, along with a mixed workload constructed from them following DeepSeek-V3.2 [20]. Table 1 summarizes the domains and datasets used in our experiments. During rollout, we set the maximum response length to 8K tokens and generate four samples per prompt using GRPO [24], with temperature 1.0 and top- p 1.0. For all domains, we use an EP size of 32, a global batch size of 1024, and 32 micro-batches.

Figure 4 illustrates the imbalance pattern dynamics, including per-expert load share and skewness, across consecutive micro-batches of Qwen3-235B-A22B [7]. Across different domains, the same color denotes the same expert. We define the load skewness of a micro-batch as the ratio between the maximum expert load and the average expert load over all experts in that micro-batch. Figure 4 reveals fluctuating load imbalance in two complementary ways. First, the load share of an individual expert varies substantially from one micro-batch to the next, indicating that the hot experts are not stable even within the same domain. Second, the skewness of the expert load distribution is dynamic. Different domains exhibit different skewness dynamics and activate different subsets of experts. When several domains are mixed for RL training, hot experts can shift rapidly across micro-batches.

To quantify the temporal fluctuation of load imbalance, we measure the intersection ratio of the top- k highest-load experts between adjacent micro-batches, defined as $\frac{|H_t^k \cap H_{t+1}^k|}{k}$, where H_t^k denotes the set of top- k highest-load experts in micro-batch t . A lower intersection ratio indicates more drastic changes in hot experts across micro-batches. Figure 5 reports the intersection ratio of the top-4 and top-8 highest-

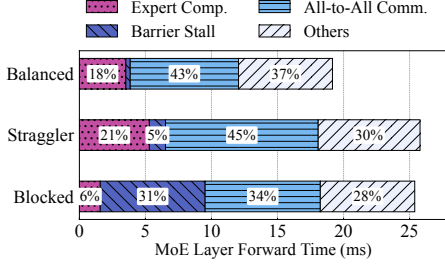


Figure 6: One MoE layer’s forward time breakdown across different micro-batches.

load experts on the mixed domain, with results averaged over all MoE layers for each model. Across three MoE models ranging from 30B to 235B parameters, the intersection ratio stays below 0.5 for most cases, indicating that more than half of the hot experts change from one micro-batch to the next.

Figure 6 illustrates how fluctuating load imbalance degrades performance. The “Blocked” and “Straggler” cases show the forward time breakdowns of the same MoE layer of Qwen3-235B-A22B on the mixed domain, measured on the same GPU for two different micro-batches. For comparison, we also evaluate a “Balanced” setting that enforces balanced routing. Relative to the Balanced setting, the Straggler case incurs substantially longer expert computation and communication time, indicating that excessive routed tokens directly create stragglers on the corresponding GPUs. By contrast, although the Blocked case has shorter local computation time, waiting still accounts for 31% of its total layer forward time because it is stalled by remote stragglers. Overall, such fluctuating imbalance patterns increase the forward time to $1.4\times$ compared to balanced routing. In real deployments, MoE training is typically parallelized with DP, PP, and EP, so such unpredictable skew can propagate beyond a single layer, creating pipeline bubbles under PP and cross-rack stragglers under DP, further amplifying the performance degradation.

2.3 Opportunity and Challenge

Opportunity: routing replay. As described in Section 1, RL offers a unique load-balancing opportunity that is absent in MoE pretraining. Because rollout and training use the same set of MoE parameters to process the same token sequence, the token-to-expert routing information of each training sample can be shared between the two stages. This enables the training system to observe the exact expert demand of upcoming micro-batches, rather than predicting it from recent history, allowing proactive and fine-grained load balancing. Recent industrial practice [20, 28, 29] has also adopted routing replay in MoE RL training to improve training stability. Moreover, RL training includes post-rollout processing before training [42, 43], whose time budget is orders of magnitude larger than the runtime decision window in pretraining, typically only tens of microseconds for each micro-batch at one layer. This additional time budget gives the system substantially more time to exploit routing replay.

Challenge: practical load-balancing planning. Knowing the exact routing of a micro-batch does not translate into a practical load-balancing plan. For each MoE layer of each micro-batch, the system must decide how to redistribute load through expert reordering, expert replication, or both, while respecting GPU memory limits and communication costs. The resulting problem is a large combinatorial optimization that recurs throughout training. Even simplified variants are already NP-hard. If expert replication is disabled and memory constraints are ignored, the problem reduces to makespan minimization on identical machines ($P||C_{\max}$) [30]. With replication and capacity constraints, it further contains replica placement decisions similar to the facility location problem [44]. Therefore, the key challenge is not merely to exploit routing replay, but to translate it into a practical load-balancing plan that remains effective at micro-batch granularity under realistic system constraints and time budgets.

Key insight: timescale decomposition. The key insight is that, under cross-node EP, expert reordering and expert replication should operate at different timescales. Although both can mitigate load imbalance, expert reordering is poorly suited to micro-batch-level fluctuating imbalance. Because hot experts may shift across nodes from one micro-batch to the next, reordering typically requires coordinated placement changes across the EP group to preserve per-GPU memory balance. This makes reordering a global operation whose cross-node coordination cost can outweigh its benefit when invoked too frequently. In contrast, expert replication fits the micro-batch timescale better: it adds capacity only to hot experts without disturbing the placement of others. It can therefore react quickly to transient load spikes using high-bandwidth intra-node links. Accordingly, expert reordering should be amortized across training batches for coarse-grained cross-node balancing, while expert replication should be applied at micro-batch granularity for fine-grained local adaptation.

3 ReLibra Overview

We propose ReLibra, an MoE RL training system that performs fine-grained load balancing at micro-batch granularity to address fluctuating load imbalance. Figure 7 shows the overall architecture of ReLibra.

Control Plane. The control plane maintains a global view of the RL training workflow. It receives the token-to-expert routing information of each sample during rollout and uses it to solve fine-grained load-balancing problems. Notably, ReLibra overlaps this optimization with post-rollout processing, such as reward computation and KL-divergence evaluation, so its solving time stays off the critical path.

Load-Balancing Solver. As ReLibra places expert reordering and replication at inter- and intra-batch timescales, the load-balancing solver decomposes the original problem into two sub-problems: inter-batch expert reordering and intra-batch

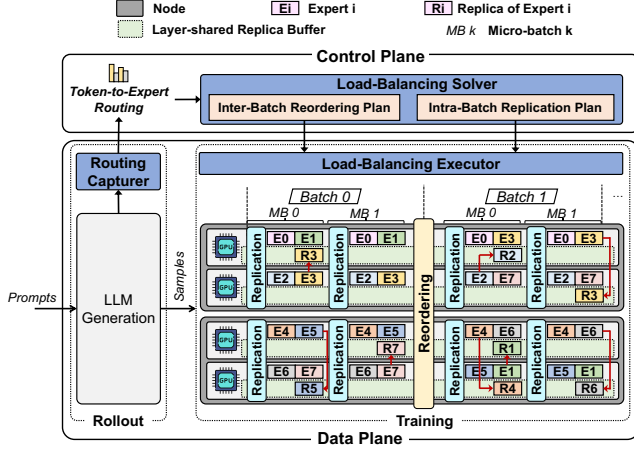


Figure 7: Overview of ReLibra.

expert replication. Inter-batch expert reordering aims to balance GPU loads across the EP group over an entire training batch. Intra-batch expert replication determines the replica count, placement, and token distribution among replicas for each hot expert. Once the solver finishes, the control plane forwards the resulting plans to the data plane for execution.

Data Plane. The data plane is responsible for collecting routing information and executing load-balancing decisions.

Routing Capturer. During rollout, the model generates response tokens in an auto-regressive, token-by-token manner. This component records the routed experts for each token at every generation step. Once generation is complete, it aggregates the token-level routing decisions for each sample and forwards them to the control plane.

Load-Balancing Executor. The load-balancing executor receives the control plane’s plans before training begins. For each training batch, it first reorders experts across the EP group for coarse-grained cross-node balancing. It then dynamically replicates experts within each node at micro-batch granularity to absorb load spikes through high-bandwidth intra-node links. To support efficient replication, the executor manages a lightweight, layer-shared replica buffer and performs two operations: transferring hot-expert parameters to destination GPUs during the forward pass and aggregating replica gradients back during the backward pass. By overlapping communication with attention computation and reusing the buffer, the load-balancing executor performs both operations with negligible overhead.

4 Inter-Batch Expert Reordering

ReLibra optimizes the inter-batch expert reordering plan across the EP group, typically spanning multiple nodes. In this section, we first formally define the inter-batch load-balancing problem. Then we propose a swap-based simulated annealing algorithm, which efficiently converges to a balanced solution and can further be used to improve data locality.

Symbol	Description
$\mathcal{G}$	Set of GPUs in the EP group
E, E_g	Set of all experts and experts on GPU g
$x_{j,e}$	Number of tokens routed to expert e from GPU j
$\tau(j, e)$	Source-destination type of tokens from GPU j to expert e
$y_{j,e,g}$	Fraction of tokens routed to expert e from GPU j to g
$z_{e,g}$	Binary indicator of whether e ’s replica can be placed on g
L_g^{comp}	GPU g ’s computation load
$C_g^{\text{nvlink,tx/rx}}$	GPU g ’s NVLink send/receive load
$C_g^{\text{rdma,tx/rx}}$	GPU g ’s RDMA send/receive load
$T_g^{\text{comp/comm}}$	MoE computation/communication time on GPU g
T_{MoE}	Execution time of the MoE module

Table 2: Key notations in the design.

4.1 Problem Formulation

Inter-batch expert reordering aims to balance the accumulated load over the entire training batch according to the aggregated routing information. For each MoE layer, ReLibra jointly considers loads of both expert computation and all-to-all communication to minimize the overall execution time of the MoE module. Table 2 lists the key notations.

From token-to-expert routing decisions to computation and communication loads. In line with prior work [17, 25, 45], we measure load in number of tokens. Since each GPU is an execution unit of the MoE module, we first demonstrate how to derive the computation and communication loads from the token-to-expert routing. For each source GPU j and expert e , let $x_{j,e}$ denote the number of tokens originally located on GPU j (i.e., for attention computation) and routed to expert e . We assume each expert resides entirely on a single GPU, rather than being sharded across multiple GPUs, which is common practice in recent fine-grained MoE models [6, 8, 9]. Let $\mathcal{G}$ be the set of GPUs in the EP group. Let E_g be the set of experts hosted by GPU g . The total load of GPU g is

$$L_g^{\text{comp}} = \sum_{e \in E_g} l_e = \sum_{e \in E_g} \sum_{j \in \mathcal{G}} x_{j,e}, \quad (1)$$

where l_e is the load of expert e hosted by GPU g .

Communication load is more complex to characterize. To better support the collective communication patterns (e.g., all-reduce) prevalent in LLM training, industrial systems have widely adopted rail-optimized network topologies [46, 47]. Figure 8 shows an example of two nodes with two GPUs each. In such a topology, GPUs and NICs with the same index are attached to the same PCIe switch and form a rail, and NICs on the same rail across nodes are connected by a leaf switch. Consequently, communication in MoE training exhibits three patterns: (i) intra-node communication over NVLink, (ii) inter-node same-rail communication with GPUDirect RDMA [48], and (iii) inter-node cross-rail communication, which first traverses NVLink to reach a GPU on the target rail and then uses RDMA for inter-node transfer [49, 50].

Given the rail-optimized topology, we classify each routed token by the relative location between its source GPU and

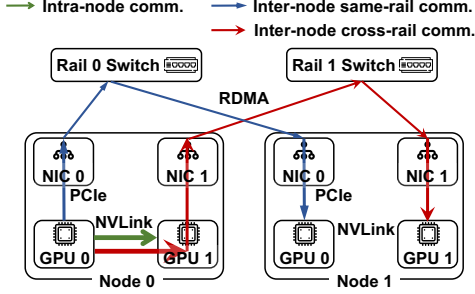


Figure 8: Rail-optimized topology for MoE training. The spine switch is omitted for brevity.

the GPU hosting its destination expert. Let $\tau(j, e)$ denote the source-destination type for tokens on GPU j routed to expert e , where $\tau(j, e) \in \{\text{loc}, \text{nv}, \text{sr}, \text{cr}\}$ corresponds to local memory copy, intra-node NVLink transfer, inter-node same-rail RDMA transfer, and inter-node cross-rail transfer with an extra NVLink relay, respectively. The source-destination type determines the communication load injected into NVLink and RDMA. For GPU g , the NVLink load during dispatch can be formulated as:

$$C_g^{\text{nvlink}, \text{tx}} = \sum_{e: \tau(g, e) \in \{\text{nv}, \text{cr}\}} x_{g, e}, \quad (2)$$

$$C_g^{\text{nvlink}, \text{rx}} = \sum_{e \in E_g} \sum_{j: \tau(j, e) \in \{\text{nv}, \text{cr}\}} x_{j, e}, \quad (3)$$

where tx and rx denote the send and receive directions, respectively. The nv and cr types contribute as direct and relayed intra-node traffic, respectively. Similarly, the RDMA load can be formulated by replacing the nv with the sr type in the above equations. For the combine phase, we only need to update each token's source GPU to the destination GPU in the dispatch phase, because combine sends tokens back to their original locations.

From loads to MoE module execution time. We then use the computation and communication loads to estimate the execution time of the MoE module, including expert computation and all-to-all communication, for the entire EP group, which we aim to minimize. For computation, given the model's hidden size h and intermediate size h' , processing one token by one expert requires approximately $6hh'$ floating-point operations ($2hh'$ for each of the three GEMMs), ignoring element-wise activation and gating overhead for simplicity. Let F denote the effective compute capability (FLOPS) of one GPU. The computation time on GPU g is

$$T_g^{\text{comp}} = \frac{6hh' L_g^{\text{comp}}}{F}. \quad (4)$$

For communication, given the effective bandwidth of NVLink and RDMA as B_{nvlink} and B_{rdma} , respectively, the communi-

Algorithm 1 Swap-based simulated annealing

Require: Experts E , GPUs $\mathcal{G}$, routing matrix $\{x_{j,e}\}$, seed set $\mathcal{S}$, cooling rate γ , termination threshold ε

Ensure: Expert reordering plan Π^*

```

1:  $\Pi_0 \leftarrow \text{GREEDYLPT}(\{x_{j,e}\}, \mathcal{G})$  ▷ initial state
2: for all  $s \in \mathcal{S}$  in parallel do
3:    $\Pi \leftarrow \Pi_0; \Pi_s^* \leftarrow \Pi_0$ 
4:    $T \leftarrow \tilde{T}_{\text{MoE}}(\Pi); T_s^* \leftarrow T; \theta \leftarrow T$ 
5:   while  $\theta > \varepsilon$  do
6:      $(e_a, e_b) \leftarrow \text{RANDOMSAMPLE}(E)$ 
7:      $\Pi' \leftarrow \text{SWAP}(\Pi, e_a, e_b)$  ▷ incremental state update
8:      $T' \leftarrow \tilde{T}_{\text{MoE}}(\Pi')$  ▷ objective smoothing
9:      $\Delta \leftarrow T - T'$ 
10:    if  $T' < T$  or  $\text{RAND}(0, 1) < e^{-\Delta/\theta}$  then
11:       $\Pi \leftarrow \Pi'; T \leftarrow T'$ 
12:    if  $T < T_s^*$  then
13:       $\Pi_s^* \leftarrow \Pi; T_s^* \leftarrow T$ 
14:     $\theta \leftarrow \gamma\theta$ 
15: return  $\arg \min_{\Pi \in \{\Pi_s^* | s \in \mathcal{S}\}} \tilde{T}_{\text{MoE}}(\Pi)$ 
```

cation time on GPU g is

$$T_g^{\text{comm}} = \max_{\phi \in \{\text{tx}, \text{rx}\}} \left\{ \max \left\{ \frac{C_g^{\text{nvlink}, \phi}}{B_{\text{nvlink}}}, \frac{C_g^{\text{rdma}, \phi}}{B_{\text{rdma}}} \right\} \right\}. \quad (5)$$

The outer maximum is due to the full-duplex nature of communication links, and the inner maximum is because existing all-to-all implementations [49, 50] optimize cross-node communication efficiency by pipelining and overlapping NVLink and RDMA transmissions. ReLibra obtains F , B_{nvlink} , and B_{rdma} values through profiling and interpolation as prior work does [51–54]. Therefore, the MoE execution time of the EP group is

$$T_{\text{MoE}} = \max_{g \in \mathcal{G}} T_g^{\text{comp}} + \max_{g \in \mathcal{G}} T_g^{\text{comm}}. \quad (6)$$

4.2 Swap-based Simulated Annealing

An exhaustive search over expert reordering plans has a search space of $O(|E|!)$. Recent MoE models [6, 7] typically contain > 100 (e.g., 128 or 256) experts per layer, making the problem combinatorially intractable. A natural solution is a greedy longest-processing-time (LPT) [55] heuristic, which sorts experts in descending order of computation load (l_e) and places each expert on the GPU satisfying the capacity constraint and having the smallest current total load. However, the greedy method lacks a global perspective. Reordering an expert changes not only its computation load contribution, but also the relative locations between that expert and its routed tokens, thereby altering the communication volume and imbalance across GPUs. Thus, the greedy approach does not yield optimal performance, as we show in § 7.2.

Instead, we propose a swap-based simulated annealing algorithm that jointly considers computation and communication loads by building on Formula 6, searching for a better solution.

For each MoE layer, the algorithm applies the LPT heuristic as an initial state. It then repeatedly swaps two randomly chosen experts, and reevaluates the MoE execution time T_{MoE} . A new state is always accepted if it improves the objective and is otherwise accepted with a temperature-controlled probability, allowing the search to escape local minima. We set the initial temperature to the initial state’s objective value. We launch multiple annealing runs with different random seeds in parallel and keep the best final reordering plan. Algorithm 1 shows the pseudocode.

Incremental state update. The swap-based search naturally enables an incremental update scheme, which significantly reduces the cost of evaluating each swap. When two experts are exchanged, only the routing contributions associated with these two experts can change. Therefore, instead of recomputing all GPU loads from scratch, ReLibra rescans $\{x_{j,e}\}$ only for the swapped experts and incrementally updates the affected computation and communication loads. The time complexity of this incremental update is $O(|\mathcal{G}|)$.

Objective smoothing. The exact objective T_{MoE} defined above contains nested max operators, which create large plateaus in the search landscape: a swap may improve several near-bottleneck GPUs but leave the current maximum unchanged. To make the objective continuous and differentiable, we replace each max with a log-sum-exp (LSE) surrogate: $\text{LSE}_\beta(z_1, \dots, z_n) = \frac{1}{\beta} \log \sum_{k=1}^n e^{\beta z_k}$, where a larger β yields a closer approximation to max. We then smooth the objective function as $\tilde{T}_{\text{MoE}} = \text{LSE}_\beta(\tilde{T}_g^{\text{comp}}, g \in \mathcal{G}) + \text{LSE}_\beta(\tilde{T}_g^{\text{comm}}, g \in \mathcal{G})$ and smooth the inner max of communication time as well. This smoothed objective provides more optimization signals. Swaps that reduce near-critical loads can still decrease $\tilde{T}_{\text{MoE}}$ even when the exact maximum does not change. We compare the performance of different β values and choose $\beta = 20$ for ReLibra.

Optimizing data locality. Cross-node EP is often combined with data parallel attention [6] to increase the batch size and reduce TP communication. Under this setup, each GPU hosts a replica of the attention module, so each sample can in principle be processed by any GPU. This flexibility creates an additional optimization opportunity: assigning a sample to a GPU that hosts more of its frequently activated experts improves locality and reduces MoE communication loads.

Given an expert reordering plan Π^* , ReLibra performs a second round of swap-based simulated annealing to optimize sample placement. This stage uses the same objective as expert reordering, but replaces expert swaps with sample swaps. Swapping two samples changes their source GPUs and therefore changes the resulting communication load and communication time. The final output is a locality-aware sample-to-GPU assignment that further reduces cross-GPU MoE traffic.

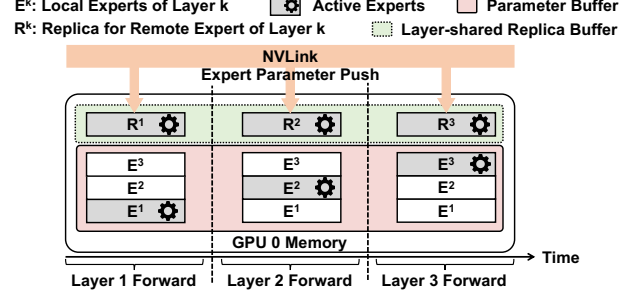


Figure 9: Layer-shared replica buffer for fine-grained intra-batch replication.

5 Intra-Batch Expert Replication

At the intra-batch timescale, ReLibra dynamically replicates experts within a node based on the inter-batch expert reordering plan to absorb micro-batch-level load fluctuations. This design poses two challenges. On the systems side, expert replication must operate at micro-batch granularity without incurring excessive memory or synchronization overhead. On the algorithmic side, once an expert has one or more replicas, the system must determine the number of replicas, their placement, and the token assignment among them. Existing solutions [17, 25] use fixed replication plans at global-batch granularity and thus sidestep these challenges. In this section, we address these challenges with a holistic design for efficient intra-batch expert replication.

5.1 Layer-Shared Replica Buffer

A straightforward replication design would reserve replica slots for every MoE layer, so that replicas can be materialized immediately whenever a layer is executed. However, the forward pass proceeds layer by layer, and only the current layer’s expert parameters are accessed at any given time. ReLibra exploits this property with a layer-shared replica buffer: each GPU allocates only a small number of replica slots, stores replicas for one layer at a time, and reuses the same buffer across all layers. Figure 9 illustrates this design with an example where each GPU can host at most one expert replica. As the forward pass progresses from Layer 1 to Layer 3, the resident experts of all layers remain in place, while the replica buffer is reused to hold the replicas required by each layer in turn. Specifically, the buffer is filled with the replicas needed by Layer 1 before expert computation, then overwritten with those for Layer 2, and so on for subsequent layers.

Memory overhead. The key advantage of a layer-shared buffer is its negligible memory overhead. Suppose each GPU stores M experts under EP in each of L MoE layers, and up to r replicas may be placed on the GPU for any single layer. A naive design that reserves a separate replica buffer for every layer requires Lr replica slots in total. In contrast, ReLibra allocates only one shared buffer with r slots and reuses it across all layers, reducing the additional memory overhead

by one to two orders of magnitude for common MoE models with tens of layers. Since **expert parameters dominate replica memory consumption**, this reduction makes micro-batch-level replication practical for deep MoE models.

Synchronization overhead. Leveraging routing replay, ReLibra makes load-balancing decisions before the current micro-batch is executed, determining which experts need to be replicated for each layer and on which GPUs these replicas should be placed. At runtime, ReLibra uses a push-based SPMD scheme to populate the shared replica buffer. During the forward pass, each GPU independently pushes local experts to their required destination GPUs before the replicas are actually used. During the backward pass, the gradients produced by the replicas of this micro-batch are pushed back and accumulated on the source GPUs. Because replication is confined within a node, this synchronization uses only high-bandwidth intra-node links. ReLibra further overlaps the synchronization operations with attention computation, avoiding any interference with the critical path.

5.2 Micro-batch-level Replication Planning

MILP formulation. For each expert e of an MoE layer in a given micro-batch, ReLibra constructs a replication plan with two coupled components. The first component is replica placement: determining whether e should be replicated and, if so, on which GPUs within the local node the replicas should be placed. The second component is token splitting: determining how the tokens routed to e should be distributed across the original expert and its replicas. These two components must be optimized jointly. Replica placement determines the available locations for routed tokens, while token splitting determines whether a placement actually reduces the peak computation and communication loads.

We now formulate micro-batch-level replication planning for one MoE layer of one micro-batch as a mixed-integer linear program. The input is the routing matrix $\{x_{j,e}\}$ after inter-batch expert reordering. Let $\mathcal{G}(e) \subseteq \mathcal{G}$ denote the GPUs in the same node as expert e 's home GPU, i.e., the candidate GPUs on which replicas of e may be placed. We introduce a binary placement variable $z_{e,g} \in \{0, 1\}$, where $z_{e,g} = 1$ means tokens of expert e are allowed to be served by GPU g . For the home GPU that already hosts e , $z_{e,g}$ is fixed to 1; for other candidate GPUs, $z_{e,g}$ represents the replica placement decision. We also introduce a continuous variable $y_{j,e,g} \in [0, 1]$, denoting the fraction of tokens on source GPU j that are routed to expert e and finally served by GPU g . Under this definition, the actual number of tokens assigned from source GPU j to GPU g for expert e is $x_{j,e}y_{j,e,g}$. Using the same time symbols as in the inter-batch reordering problem for the MoE execution time of the micro-batch, the intra-batch replication

Algorithm 2 Incremental greedy heuristic

Require: Experts E and reordering plan Π , GPUs $\mathcal{G}$, routing matrix $\{x_{j,e}\}$, candidate GPU sets $\{\mathcal{G}(e)\}$, replica slot limit r
Ensure: Replica placement plan z^* and token splitting plan y^*

- 1: $z_{e,g} \leftarrow 1$ iff $e \in E_g$, otherwise 0
- 2: $y_{j,e,g} \leftarrow 0$ for all j, e, g ▷ initialize without replica
- 3: **while** $\exists g$ with available replica slot **do**
- 4: $g_b \leftarrow \arg \max_g T_{MoE,g}(\Pi, y, z)$ ▷ bottleneck GPU
- 5: $e^* \leftarrow \arg \max_e T_{g,e}(E_g, y, z)$ ▷ bottleneck expert
- 6: $g_t \leftarrow \arg \min_{g \in \mathcal{G}(e^*) \wedge z_{e^*,g}=0 \wedge \text{HAS_SLOT}(g)} T_{MoE,g}(\Pi, y, z)$ ▷ least-load GPU
- 7:
- 8: **if** no feasible g_t exists **then**
- 9: **break**
- 10: $z_{e^*,g_t} \leftarrow 1$
- 11: $y \leftarrow \text{SOLVETOKENSPLITLP}(\{x_{j,e}\}, z)$
- 12: **return** (z, y)

planning problem is formulated as:

$$\min_{y_{j,e,g} \in [0,1], z_{e,g} \in \{0,1\}} T_{MoE} = T^{\text{comp}}(y, z) + T^{\text{comm}}(y, z), \quad (7)$$

$$\text{s.t.} \quad \sum_{g \in \mathcal{G}(e)} y_{j,e,g} = 1, \quad \forall j, e \text{ with } x_{j,e} > 0, \quad (8)$$

$$y_{j,e,g} \leq z_{e,g}, \quad \forall j, e, g \in \mathcal{G}(e), \quad (9)$$

$$z_{e,g} = 1, \quad \forall g, \forall e \in E_g, \quad (10)$$

$$\sum_{e \in E \setminus E_g: g \in \mathcal{G}(e)} z_{e,g} \leq r, \quad \forall g, \quad (11)$$

where Constraint 8 enforces that the token fractions for each non-empty routing entry (j, e) sum to 1, so the split induced by $y_{j,e,g}$ fully partitions the original token count $x_{j,e}$. This relation also allows any objective originally expressed using routed-token counts to be rewritten with the split variables $x_{j,e}y_{j,e,g}$, and hence as a function of the decision variables $y_{j,e,g}$ and $z_{e,g}$. Constraint 9 couples token splitting with replica placement, ensuring that a positive fraction can be assigned to GPU g only when the original expert or one of its replicas is present there. Constraint 10 fixes the placement of the home GPU. Constraint 11 captures the per-GPU replica buffer limit introduced in § 5.1. Since the objective contains only linear terms and max operators with respect to the decision variables, it can be converted into an MILP problem by introducing auxiliary variables to linearize the maxima.

Solving MILP. Although the MILP formulation is compact, solving it for every micro-batch remains computationally expensive. This difficulty arises from two sources. First, replica placement introduces binary variables over expert-GPU pairs, causing the search space to grow combinatorially with the number of candidate experts and available replica slots. Second, general-purpose MILP solvers such as SCIP [56] incur substantial branch-and-bound overhead and often fail to produce high-quality solutions within the time budget (e.g., millisecond-level per micro-batch) in our setting.

We propose an incremental greedy heuristic to avoid direct MILP solving. The heuristic incrementally constructs the replica placement and re-optimizes token splitting after each placement decision. Algorithm 2 shows the detailed procedure. It starts from the no-replica state and then optimizes the replication plan in an iterative manner. In each iteration, it first identifies the current bottleneck GPU g_b using computation and communication loads. Among the experts on g_b , it selects the one contributing the largest load. Next, one replica is added to the feasible GPU with the smallest current load. Finally, given a new replica placement z , it re-solves the token-splitting subproblem using linear programming to obtain the optimal split ratios y . This process repeats until no replica slot remains.

6 Implementation

We implement ReLibra based on Slime [57] for RL workflow orchestration and Megatron-LM [58] for MoE training with 14K lines of Python, C++, and CUDA. We use SGLang [59] as the rollout backend.

Routing capturer. Built on top of SGLang, the routing capturer exposes routing information after each response generation. We store the routing metadata in a small GPU buffer (tens of MB) and asynchronously transfer it to host memory, incurring negligible overhead during rollout.

Load-balancing solver. We implement both the inter-batch reordering and intra-batch replication solvers in C++ with OpenMP [60]. Each thread solves one independent optimization task: one layer with one simulated-annealing seed for inter-batch reordering, or one layer with one micro-batch for intra-batch replication.

Layer-shared replica buffer. We implement layer-shared replica buffer synchronization with a persistent asynchronous TMA kernel [61] based on the NVIDIA Hopper architecture. To minimize interference with attention computation, we limit the synchronization kernel to occupy only a small number of streaming processors (SMs), leaving most SMs available for compute-intensive attention and thereby overlapping synchronization with computation while minimizing contention.

7 Evaluation

In this section, we first compare the overall performance of ReLibra against state-of-the-art MoE training systems (§ 7.1). To further validate ReLibra’s effectiveness, we strengthen history-based load-balancing methods by providing oracle loads from routing replay. We then perform ablation studies to quantify the contribution of each component of ReLibra and compare them with strawman baselines (§ 7.2). Next, we demonstrate the near-optimal imbalance reduction capability of ReLibra (§ 7.3). Finally, we investigate the system overhead of ReLibra (§ 7.4).

Model	#MoE Layers	#GPUs	DP	PP	EP
Qwen3-30B-A3B	48	64	4	1	16
GLM4.5-106B-A12B	45	64	1	2	32
Qwen3-235B-A22B	94	160	1	5	32

Table 3: Parallelism strategies for different models.

Testbed. We conduct experiments on a 20-node cluster, where each node has eight NVIDIA Hopper GPUs and 1.5 TB of host memory. Nodes are connected by a rail-optimized InfiniBand RDMA network with an aggregate bandwidth of 8×400 Gbps per node, while intra-node GPU communication is facilitated by 900 GB/s NVLink. Our software stack includes PyTorch 2.8.0, CUDA 12.6 (driver 535.161.08), FlashAttention-3 [62], TransformerEngine 2.7.0 [63], and DeepEP 1.1.0 [50]. The load-balancing solver in § 3 runs on a separate CPU-only node with 128 Intel Xeon CPU cores, and we use Ray [64] to dispatch solving tasks.

Workloads and models. We evaluate ReLibra on three representative RL workloads: reasoning, instruction following, and coding. The corresponding datasets and rollout hyperparameters are the same as those used in Table 1 in § 2. We also mix these datasets to better reflect production RL training workloads, as suggested by DeepSeek-V3.2 [20]. We evaluate ReLibra on Qwen3-30B-A3B [7], GLM4.5-106B-A12B [65], and Qwen3-235B-A22B [7], which span diverse model scales and come from different vendors. These models have 128 experts per layer and choose top-8 experts for each token.

Baselines. We compare ReLibra with the following baselines.

- **Megatron-LM** [58] is a widely used, open-source LLM training framework that supports hybrid parallelism and integrates various MoE optimizations, including DeepEP [50] and GroupedGEMM for efficient all-to-all communication and expert computation, respectively.
- **PopFetcher** [17] predicts which experts in the current layer are most likely to be activated for the current training batch based on recent cross-layer activation patterns, then prefetches them for replication and overlaps the prefetching with attention computation. We tune its window size to achieve the best performance in our experiments.
- **EPLB⁺** strengthens EPLB [25] by replacing historical load observations with oracle loads from routing replay. EPLB, proposed by DeepSeek-V3 [6], is a history-based MoE load-balancing method that uses greedy algorithms to optimize expert reordering and replica placement at global-batch granularity, considering only computation load.
- **LPLB⁺** strengthens LPLB [45] in the same way as EPLB⁺ by providing oracle loads. LPLB is an EPLB variant with dynamic token splitting for each micro-batch via a GPU-based linear programming solver. To reduce solving overhead, it restricts each expert to at most one replica.
- **Balanced** is a system-level ideal baseline that randomly routes tokens to all experts to achieve optimal load balancing. However, it does not preserve the model’s original

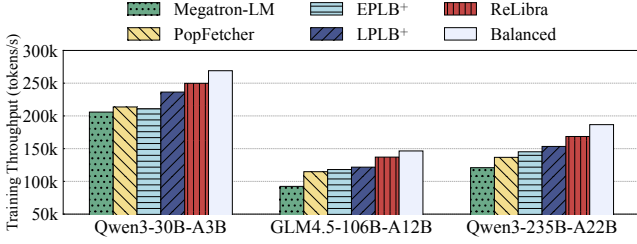


Figure 10: Training throughput on the mixed dataset with different models.

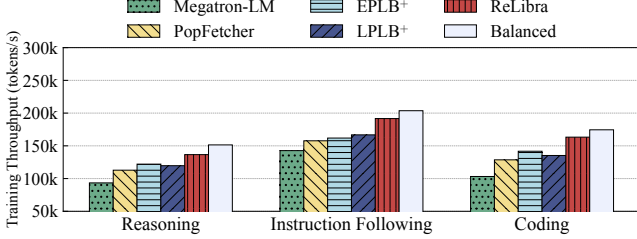


Figure 11: Training throughput on different datasets with Qwen3-235B-A22B.

routing decisions and thus serves only as a reference for the best-case system efficiency.

For a fair comparison, we integrate all methods into Megatron-LM and use the same parallelism strategy. We further employ the same layer-shared replication buffer, the number of replication slots (two replica slots per GPU in our experiments), and the synchronization overlapping mechanism for all baselines using expert replication. This ensures that the performance gains of ReLibra arise from its load-balancing techniques rather than other system optimizations.

Metrics. ReLibra targets the fluctuating load-imbalance problem during training and is orthogonal to rollout optimizations. We therefore use training throughput, defined as the throughput of the training stage, as our primary metric. In the case study (§ 7.3), we also report rank-level skewness, defined as the ratio of the maximum to the average number of tokens processed across GPUs, to quantify the degree of load imbalance.

7.1 Overall Performance

We first benchmark ReLibra against baselines on different models and datasets. We deploy the three models with different parallelism strategies based on their parameter scales. Specifically, we set the EP size to 16 for the 30B model and 32 for the other two models, and we employ data parallel attention as industrial practice suggests [6] to reduce TP communication overhead. Table 3 details the configurations. We apply the same global batch size, 1024, for all methods under all settings. Figure 10 and Figure 11 present the average training throughput across various models and datasets. We summarize the results as follows.

Idx	Method	Reasoning		Mixed	
		Normalized Throughput	Δ	Normalized Throughput	Δ
1	Megatron-LM	1		1	
2	(1) with inter-batch reordering	1.12	+12%	1.15	+15%
3	(2) with intra-batch replication	1.46	+34%	1.39	+24%

Table 4: Throughput improvement breakdown of inter-batch and intra-batch load balancing when training Qwen3-235B-A22B.

- ReLibra outperforms all baselines, achieving speedups of $1.21\text{--}1.58\times$ over Megatron-LM, $1.17\text{--}1.27\times$ over PopFetcher, and $1.06\text{--}1.21\times$ over EPLB⁺ and LPLB⁺, respectively. The gains come from ReLibra’s ability to capture fluctuating load imbalance at micro-batch granularity and adapt its load-balancing decisions accordingly. In contrast, PopFetcher depends on imperfect load prediction, while EPLB⁺ and LPLB⁺ use static expert reordering and replication plans throughout the training batch. As a result, even with oracle loads, they cannot respond to fine-grained imbalance fluctuations.
- ReLibra achieves comparable performance to Balanced, reaching 90%–94% of its training throughput. This is enabled by its hierarchical design, which uses inter-batch expert reordering for coarse-grained balancing and intra-batch expert replication for fine-grained adaptation, leading to near-balanced execution for each micro-batch.
- ReLibra is effective across RL domains, model scales, and parallelism strategies. LPLB⁺ performs slightly worse than EPLB⁺ on reasoning and coding, but better on instruction following and mixed workloads. This is because LPLB⁺ uses linear programming to dynamically split tokens across replicas, while restricting each expert to at most one replica to keep the search space manageable. As a result, it is more effective for highly fluctuating imbalance but less effective for strongly skewed imbalance. ReLibra avoids this trade-off by replaying routing and computing replication plans before training, while overlapping solver execution with post-rollout processing so that it adds no extra training time.

7.2 Ablation Study

In this subsection, we perform ablation studies to quantify the contribution and verify the effectiveness of each component of ReLibra. We use Qwen3-235B-A22B as the test model and evaluate it on the reasoning and mixed datasets.

We first perform an ablation study that incrementally enables inter-batch and intra-batch load-balancing techniques to isolate their contributions. Table 4 reports the resulting training throughput breakdown. Enabling only inter-batch expert reordering yields speedups of 12% and 15% on the reasoning and mixed datasets, respectively. Adding intra-batch expert replication further improves throughput by 34% and 24%, respectively. This is because inter-batch reordering distributes hot experts more evenly across nodes, thereby creating more opportunities for effective intra-node replication to absorb micro-batch-level load fluctuations. We then deep dive into each component of ReLibra.

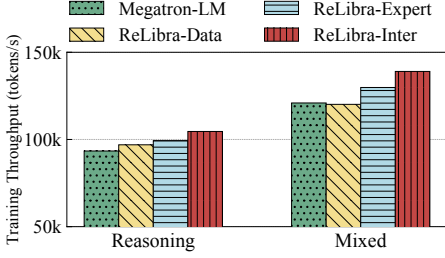


Figure 12: Performance of inter-batch expert reordering.

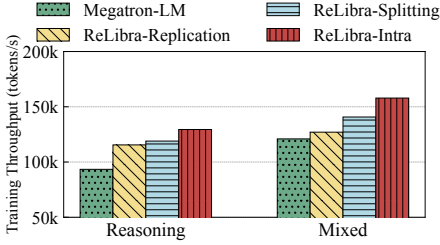


Figure 13: Performance of intra-batch expert replication.

Inter-batch expert reordering. To evaluate ReLibra’s inter-batch expert reordering, we compare it (referred to as ReLibra-Inter) against two greedy reordering baselines:

- **ReLibra-Expert** uses the LPT algorithm to place experts on GPUs, providing the initial state for our swap-based simulated annealing algorithm.
- **ReLibra-Data** uses the LPT algorithm to assign data samples to GPUs for data locality optimization. Specifically, it greedily assigns samples from long to short to the GPU with the lowest estimated MoE communication load according to Formula 5. This assignment serves as the initial solution for the second round of swap-based simulated annealing for data-locality optimization.

Figure 12 shows that ReLibra-Inter achieves $1.05\text{--}1.16\times$ higher training throughput than both baselines, demonstrating its effectiveness in balancing load across GPUs while improving data locality. The results also show that although greedy LPT improves performance over the standard EP plan in Megatron-LM, our swap-based simulated annealing algorithm converges to better solutions.

Intra-batch expert replication. To evaluate ReLibra’s intra-batch expert replication, we compare it (referred to as ReLibra-Intra) against two oracle baselines, each optimizing one component of the replication plan, namely replica placement or token splitting:

- **ReLibra-Replication** re-optimizes replica placement for each micro-batch using EPLB⁺’s greedy replication algorithm with oracle loads, while retaining EPLB⁺’s round-robin token splitting.
- **ReLibra-Splitting** re-optimizes token splitting for each micro-batch by removing the replica count limit in LPLB⁺ and solving offline linear programs with oracle loads.

Figure 13 shows that ReLibra-Intra improves training throughput by $1.12\text{--}1.24\times$ over ReLibra-Replication and by 1.09--

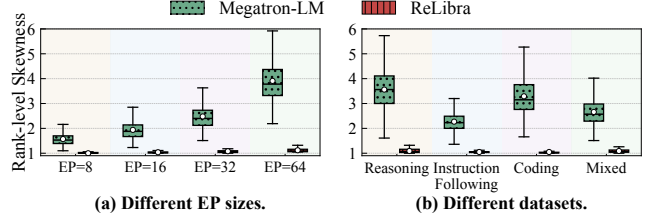


Figure 14: Rank-level skewness under different EP sizes and datasets.

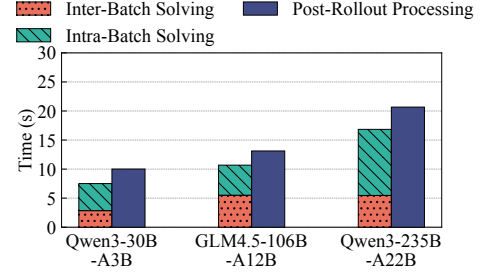


Figure 15: Solving time breakdown of ReLibra.

$1.12\times$ over ReLibra-Splitting, demonstrating the benefit of formulating replication planning as a holistic MILP problem. ReLibra-Splitting consistently outperforms ReLibra-Replication because token splitting provides finer-grained control over load distribution. Unlike replica placement, which adjusts capacity only in discrete steps, token splitting directly redistributes tokens across available replicas and can therefore adapt more precisely to fluctuating load imbalance.

7.3 Case Study

In this subsection, we evaluate the imbalance reduction capability of ReLibra across different EP sizes and datasets. Due to resource constraints, we use Qwen3-30B-A3B for the EP-size study. Figure 14(a) shows the distribution of rank-level skewness for EP sizes from 8 to 64, where skewness is defined as the ratio of the maximum to the average MoE token count across GPUs. According to Formula 1 and 2, this metric directly reflects the degree of load imbalance. As EP size increases, Megatron-LM exhibits more severe imbalance because each GPU hosts fewer experts, making the load distribution less even. In contrast, ReLibra maintains an average skewness close to 1.0 (perfectly balanced) across all EP sizes, with values of 1.00, 1.03, 1.06, and 1.08, respectively. Figure 14(b) further reports skewness on Qwen3-235B-A22B across different datasets, where ReLibra achieves average values of $1.02\text{--}1.07$, demonstrating its effectiveness across RL domains with diverse imbalance patterns.

7.4 System Overhead

Solving time. We first measure the solving time of ReLibra’s inter-batch and intra-batch load-balancing solvers, which run on a separate CPU node during post-rollout processing before training starts. Figure 15 shows that the total solving time

Model	Layer-shared Replica Buffer	Model Parameter	Ratio
Qwen3-30B-A3B	18 MiB	6.26 GiB	0.28%
GLM4.5-106B-A12B	22 MiB	16.64 GiB	0.13%
Qwen3-235B-A22B	72 MiB	4.35 GiB	1.62%

Table 5: Additional parameter memory usage of the layer-shared replica buffer per GPU.

is fully hidden behind post-rollout processing, and thus does not lie on the critical path of RL training for all three models. Qwen3-235B-A22B incurs longer intra-batch solving time than the other two models because it contains more MoE layers. Since optimization problems are independent across layers, this overhead can be further reduced with more CPU cores and higher parallelism.

Expert reordering overhead. We then measure the overhead of inter-batch expert reordering. Because expert reordering operates at the inter-batch timescale, its cost is amortized over the entire training batch. With a global batch size of 1024, the average expert reordering time accounts for only 1.4%–2.1% of the execution time of the training batch across the three models, indicating negligible overhead.

Expert replication overhead. Finally, we measure the overhead of intra-batch expert replication in two aspects: (i) the additional memory usage of the layer-shared replica buffer, and (ii) the synchronization overhead for dynamic expert replication. Table 5 reports the total parameter memory per GPU and the additional memory per GPU used for expert replication in the layer-shared replica buffer for all models. As expected, the extra memory is negligible, accounting for only 0.13%–1.62% of the total parameter memory. The additional gradient memory is identical, since each replicated parameter corresponds to one gradient. This small overhead arises because ReLibra stores replicas for only one layer at a time and reuses the same buffer across layers.

Figure 16 shows the synchronization overhead of dynamic expert replication during forward and backward. In the forward pass, each GPU pushes local experts to the GPUs hosting their replicas; in the backward pass, it sends the replica gradients back. We refer to this parameter and gradient transfer time as replica synchronization time. The figure reports, for all models, the average replica synchronization time, the attention computation time without overlap, and the attention computation time when overlapped with replica synchronization. The results show that replica synchronization is fully hidden behind attention computation and introduces negligible interference to its execution time. This is because ReLibra limits the synchronization kernel to occupy only a small number of SMs, leaving most SMs available for attention.

8 Discussion

Off-policy RL. To improve data efficiency and accelerate the RL iteration, some off-policy RL algorithms either reuse the same batch of data for multiple parameter updates [26, 27] or allow rollout and training to proceed asynchronously [66, 67].

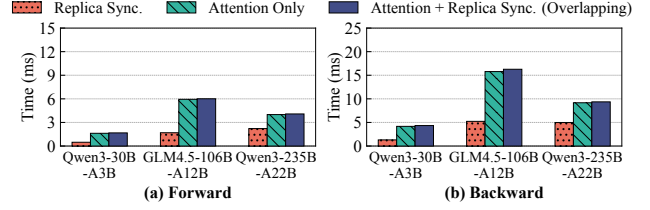


Figure 16: Synchronization overhead of ReLibra.

In these cases, rollout and training do not necessarily use the same model parameters, resulting in potentially different routing results. However, recent industrial practice [20, 28, 29] has also adopted routing replay in off-policy RL training, which substantially helps stabilize training by guaranteeing that experts are trained by the exact tokens generated by themselves.

9 Related Work

MoE training. MoE scales model capacity efficiently [2, 3, 6, 11] but faces challenges in load balancing and all-to-all communication. Existing MoE load-balancing methods, mostly designed for pretraining, use historical observations to predict future expert loads [14, 15, 17, 25, 68], making them less effective for RL training with fluctuating imbalance. Furthermore, the tight decision window in pretraining limits their strategy space [16, 18]. ReLibra addresses these problems by using routing replay to obtain exact routing information, enabling fine-grained load balancing via expert reordering and replication. For all-to-all communication, existing solutions mainly follow three directions. One line of work [50, 69, 70] uses hierarchical all-to-all to reduce cross-node traffic by exploiting bandwidth asymmetry. Another improves efficiency by overlapping communication with computation [6, 71–73] or fusing data movement with compute operators [74, 75]. A third takes a data-centric approach by moving experts closer to the data [17, 76], thereby reducing communication volume when experts are small. ReLibra is orthogonal to these approaches and can be combined with them by adapting Formula 5 to their communication patterns.

RL training. RL training is becoming increasingly important for enhancing LLM capabilities and human alignment. Existing systems generally adopt two deployment patterns: synchronous RL, where rollout and training execute on a shared GPU pool in turn [77], and asynchronous RL, which overlaps these stages across separate GPU pools to maximize hardware utilization [8, 67, 78] but at the cost of training stale samples. In the synchronous setting, optimizations like Seer [41] and TLT [40] accelerate rollout via speculative decoding [79, 80], while RLHFuse [42] minimizes GPU idle time by initiating training early during rollout. ReLibra is orthogonal to these efforts. While prior works focus on accelerating rollout or orchestrating RL workflows, ReLibra optimizes the training stage itself. ReLibra generalizes to both deployment patterns

through routing replay, consistent with recent industrial approaches for stabilizing MoE training [20, 28, 29].

10 Conclusion

We present ReLibra, an MoE RL training system that addresses fluctuating load imbalance by exploiting routing replay in the RL workflow. ReLibra decomposes load balancing into inter- and intra-batch timescales, matching expert reordering and replication to the hierarchical bandwidths of training clusters. With efficient solving algorithms and a lightweight layer-shared replica buffer, ReLibra can adapt to load imbalance at micro-batch granularity with negligible overhead. Our evaluation shows that ReLibra achieves near-optimal load balancing and improves training throughput by up to $1.6\times$ over state-of-the-art MoE training systems.

References

- [1] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, “Outrageously large neural networks: The sparsely-gated mixture-of-experts layer,” *International Conference on Learning Representations (ICLR)*, 2017.
- [2] D. Lepikhin, H. Lee, Y. Xu, D. Chen, O. Firat, Y. Huang, M. Krikun, N. Shazeer, and Z. Chen, “Gshard: Scaling giant models with conditional computation and automatic sharding,” *International Conference on Learning Representations (ICLR)*, 2021.
- [3] W. Fedus, B. Zoph, and N. Shazeer, “Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity,” *Journal of Machine Learning Research*, 2022.
- [4] “Introducing DBRX: A New State-of-the-Art Open LLM,” 2024. <https://www.databricks.com/blog/introducing-dbrx-new-state-art-open-llm>.
- [5] A. Q. Jiang, A. Sablayrolles, A. Roux, A. Mensch, B. Savary, C. Bamford, D. S. Chaplot, D. d. l. Casas, E. B. Hanna, F. Bressand, *et al.*, “Mixtral of experts,” *arXiv preprint arXiv:2401.04088*, 2024.
- [6] A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, *et al.*, “Deepseek-v3 technical report,” *arXiv preprint arXiv:2412.19437*, 2024.
- [7] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, *et al.*, “Qwen3 technical report,” *arXiv preprint arXiv:2505.09388*, 2025.
- [8] K. Team, Y. Bai, Y. Bao, Y. Charles, C. Chen, G. Chen, H. Chen, H. Chen, J. Chen, N. Chen, *et al.*, “Kimi k2: Open agentic intelligence,” *arXiv preprint arXiv:2507.20534*, 2025.
- [9] A. Chen, A. Li, B. Gong, B. Jiang, B. Fei, B. Yang, B. Shan, C. Yu, C. Wang, C. Zhu, *et al.*, “Minimax-m1: Scaling test-time compute efficiently with lightning attention,” *arXiv preprint arXiv:2506.13585*, 2025.
- [10] N. Du, Y. Huang, A. M. Dai, S. Tong, D. Lepikhin, Y. Xu, M. Krikun, Y. Zhou, A. W. Yu, O. Firat, *et al.*, “Glam: Efficient scaling of language models with mixture-of-experts,” in *International Conference on Machine Learning (ICML)*, 2022.
- [11] S. Rajbhandari, C. Li, Z. Yao, M. Zhang, R. Y. Aminabadi, A. A. Awan, J. Rasley, and Y. He, “Deepseek-moe: Advancing mixture-of-experts inference and training to power next-generation ai scale,” in *International Conference on Machine Learning (ICML)*, 2022.
- [12] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, *et al.*, “Training language models to follow instructions with human feedback,” *Advances in Neural Information Processing Systems*, 2022.
- [13] D. Guo, D. Yang, H. Zhang, J. Song, P. Wang, Q. Zhu, R. Xu, R. Zhang, S. Ma, X. Bi, *et al.*, “Deepseek-r1 incentivizes reasoning in llms through reinforcement learning,” *Nature*, 2025.
- [14] X. Nie, X. Miao, Z. Wang, Z. Yang, J. Xue, L. Ma, G. Cao, and B. Cui, “Flexmoe: Scaling large-scale sparse pre-trained model training via dynamic device placement,” *ACM SIGMOD*, 2023.
- [15] M. Zhai, J. He, Z. Ma, Z. Zong, R. Zhang, and J. Zhai, “{SmartMoE}: Efficiently training {Sparsely-Activated} models through combining offline and online parallelization,” in *USENIX ATC*, 2023.
- [16] C. Zhao, W. Wu, L. Song, and Y. Xu, “Micromoe: Fine-grained load balancing for mixture-of-experts with token scheduling,” *arXiv preprint arXiv:2511.16947*, 2025.
- [17] J. Zhang, C. Ma, X. Wang, Y. Nie, Y. Li, Y. Xu, X. Liao, B. Li, and H. Jin, “{PopFetcher}: Towards accelerated {Mixture-of-Experts} training via popularity based {Expert-Wise} prefetch,” in *USENIX ATC*, 2025.
- [18] X. Liu, Y. Wang, F. Fu, X. Xiao, H. Li, J. Li, and B. Cui, “Laer-moe: Load-adaptive expert re-layout for efficient mixture-of-experts training,” in *ACM ASPLOS*, 2026.
- [19] D. Liu, Z. Yan, X. Yao, T. Liu, V. Korthikanti, E. Wu, S. Fan, G. Deng, H. Bai, J. Chang, *et al.*, “Moe parallel folding: Heterogeneous parallelism mappings for efficient large-scale moe model training with megatron core,” *arXiv preprint arXiv:2504.14960*, 2025.

- [20] A. Liu, A. Mei, B. Lin, B. Xue, B. Wang, B. Xu, B. Wu, B. Zhang, C. Lin, C. Dong, *et al.*, “Deepseek-v3.2: Pushing the frontier of open large language models,” *arXiv preprint arXiv:2512.02556*, 2025.
- [21] Q. Yu, Z. Zhang, R. Zhu, Y. Yuan, X. Zuo, Y. Yue, W. Dai, T. Fan, G. Liu, L. Liu, *et al.*, “Dapo: An open-source llm reinforcement learning system at scale,” *arXiv preprint arXiv:2503.14476*, 2025.
- [22] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. Dal Lago, *et al.*, “Competition-level code generation with alphacode,” *Science*, 2022.
- [23] V. Pyatkin, S. Malik, V. Graf, H. Ivison, S. Huang, P. Dasigi, N. Lambert, and H. Hajishirzi, “Generalizing verifiable instruction following,” *arXiv preprint arXiv:2507.02833*, 2025.
- [24] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. Li, Y. Wu, *et al.*, “Deepseek-math: Pushing the limits of mathematical reasoning in open language models,” *arXiv preprint arXiv:2402.03300*, 2024.
- [25] “Expert Parallelism Load Balancer (EPLB),” 2025. <https://github.com/deepseek-ai/eplb>.
- [26] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, *et al.*, “Human-level control through deep reinforcement learning,” *Nature*, 2015.
- [27] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor,” in *International Conference on Machine Learning (ICML)*, 2018.
- [28] W. Ma, H. Zhang, L. Zhao, Y. Song, Y. Wang, Z. Sui, and F. Luo, “Stabilizing moe reinforcement learning by aligning training and inference routers,” *arXiv preprint arXiv:2510.11370*, 2025.
- [29] C. Zheng, K. Dang, B. Yu, M. Li, H. Jiang, J. Lin, Y. Liu, H. Lin, C. Wu, F. Hu, *et al.*, “Stabilizing reinforcement learning with llms: Formulation and practices,” *arXiv preprint arXiv:2512.01374*, 2025.
- [30] R. L. Graham, E. L. Lawler, J. K. Lenstra, and A. R. Kan, “Optimization and approximation in deterministic sequencing and scheduling: a survey,” in *Annals of discrete mathematics*, 1979.
- [31] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” *Advances in Neural Information Processing Systems*, 2017.
- [32] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, “Zero: Memory optimizations toward training trillion parameter models,” in *International Conference for High Performance Computing, Networking, Storage and Analysis*, 2020.
- [33] M. Shoenberger, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, “Megatron-lm: Training multi-billion parameter language models using model parallelism,” *arXiv preprint arXiv:1909.08053*, 2019.
- [34] Y. Huang, Y. Cheng, A. Bapna, O. Firat, D. Chen, M. Chen, H. Lee, J. Ngiam, Q. V. Le, Y. Wu, *et al.*, “Gpipe: Efficient training of giant neural networks using pipeline parallelism,” *Advances in Neural Information Processing Systems*, 2019.
- [35] D. Narayanan, A. Harlap, A. Phanishayee, V. Seshadri, N. R. Devanur, G. R. Ganger, P. B. Gibbons, and M. Zaharia, “Pipedream: Generalized pipeline parallelism for dnn training,” in *ACM SOSP*, 2019.
- [36] D. Rein, B. L. Hou, A. C. Stickland, J. Petty, R. Y. Pang, J. Dirani, J. Michael, and S. R. Bowman, “Gpqa: A graduate-level google-proof q&a benchmark,” in *First conference on language modeling*, 2024.
- [37] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, “Orca: A distributed serving system for {Transformer-Based} generative models,” in *USENIX OSDI*, 2022.
- [38] B. Wu, Y. Zhong, Z. Zhang, S. Liu, F. Liu, Y. Sun, G. Huang, X. Liu, and X. Jin, “Fast distributed inference serving for large language models,” *arXiv preprint arXiv:2305.05920*, 2023.
- [39] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [40] Q. Hu, S. Yang, J. Guo, X. Yao, Y. Lin, Y. Gu, H. Cai, C. Gan, A. Klimovic, and S. Han, “Taming the long-tail: Efficient reasoning rl training with adaptive drafter,” in *ACM ASPLOS*, 2026.
- [41] R. Qin, W. He, W. Huang, Y. Zhang, Y. Zhao, B. Pang, X. Xu, Y. Shan, Y. Wu, and M. Zhang, “Seer: Online context learning for fast synchronous llm reinforcement learning,” *arXiv preprint arXiv:2511.14617*, 2025.
- [42] Y. Zhong, Z. Zhang, B. Wu, S. Liu, Y. Chen, C. Wan, H. Hu, L. Xia, R. Ming, Y. Zhu, *et al.*, “Optimizing {RLHF} training for large language models with stage fusion,” in *USENIX NSDI*, 2025.

- [43] R. Zhu, M. Han, Y. Zhong, W. Xiao, X. Liu, and X. Jin, "Towards efficient reward service for rlvr with request-level flexibility and batch-level constraint," in *USENIX NSDI*, 2026.
- [44] G. Cornuejols, M. Fisher, and G. L. Nemhauser, "On the uncapacitated location problem," in *Annals of Discrete Mathematics*, 1977.
- [45] "Linear-Programming-Based Load Balancer (LPLB)," 2025. <https://github.com/deepseek-ai/LPLB>.
- [46] K. Qian, Y. Xi, J. Cao, J. Gao, Y. Xu, Y. Guan, B. Fu, X. Shi, F. Zhu, R. Miao, *et al.*, "Alibaba hpn: A data center network for large language model training," in *ACM SIGCOMM*, 2024.
- [47] "NVIDIA GTC: Accelerating Mixture of Experts Training With Rail-Optimized InfiniBand Networking in Crusoe Cloud," 2024. <https://www.nvidia.com/en-us/on-demand/session/gtc24-s63014/>.
- [48] "GPUDirect RDMA," 2026. <https://developer.nvidia.com/gpudirect>.
- [49] "Optimized primitives for inter-GPU communication," 2026. <https://github.com/NVIDIA/nccl>.
- [50] "DeepEP: an efficient expert-parallel communication library," 2025. <https://github.com/deepseek-ai/DeepEP>.
- [51] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, "{DistServe}: Disaggregating prefill and decoding for goodput-optimized large language model serving," in *USENIX OSDI*, 2024.
- [52] R. Zhu, Z. Jiang, C. Jin, P. Wu, C. A. Stuardo, D. Wang, X. Zhang, H. Zhou, H. Wei, Y. Cheng, *et al.*, "Megascall-infer: Efficient mixture-of-experts model serving with disaggregated expert parallelism," in *ACM SIGCOMM*, 2025.
- [53] Z. Zhang, Y. Zhong, Y. Jiang, H. Hu, J. Sun, Z. Ge, Y. Zhu, D. Jiang, and X. Jin, "Disttrain: Addressing model and data heterogeneity with disaggregated training for multimodal large language models," in *ACM SIGCOMM*, 2025.
- [54] Z. Zhang, Y. Zhong, C. Yang, C. Jin, B. Wu, X. Wei, Y. Liu, and X. Jin, "Heddl: A distributed orchestration system for agentic rl rollout," *arXiv preprint arXiv:2603.28101*, 2026.
- [55] R. L. Graham, "Bounds on multiprocessing timing anomalies," *SIAM journal on Applied Mathematics*, 1969.
- [56] S. Bolusani, M. Besançon, K. Bestuzheva, A. Chmiela, J. Dionísio, T. Donkiewicz, J. van Doornmalen, L. Eifler, M. Ghannam, A. Gleixner, *et al.*, "The scip optimization suite 9.0," *arXiv preprint arXiv:2402.17702*, 2024.
- [57] "Slime: An LLM post-training framework for RL Scaling," 2025. <https://github.com/THUDM/slime/>.
- [58] "GPU optimized techniques for training transformer models at-scale," 2025. <https://github.com/NVIDIA/Megatron-LM>.
- [59] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, *et al.*, "Sglang: Efficient execution of structured language model programs," *Advances in Neural Information Processing Systems*, 2024.
- [60] "Open Multi-Processing," 2026. <https://www.openmp.org/>.
- [61] "NVIDIA Hopper Architecture In-Depth," 2022. <https://developer.nvidia.com/blog/nvidia-hopper-architecture-in-depth/>.
- [62] J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani, and T. Dao, "Flashattention-3: Fast and accurate attention with asynchrony and low-precision," *Advances in Neural Information Processing Systems*, 2024.
- [63] "A library for accelerating Transformer models on NVIDIA GPUs, including using 8-bit floating point (FP8) precision on Hopper and Ada GPUs, to provide better performance with lower memory utilization in both training and inference.," 2026. <https://github.com/NVIDIA/TransformerEngine>.
- [64] P. Moritz, R. Nishihara, S. Wang, A. Tumanov, R. Liaw, E. Liang, M. Elibol, Z. Yang, W. Paul, M. I. Jordan, *et al.*, "Ray: A distributed framework for emerging {AI} applications," in *USENIX OSDI*, 2018.
- [65] A. Zeng, X. Lv, Q. Zheng, Z. Hou, B. Chen, C. Xie, C. Wang, D. Yin, H. Zeng, J. Zhang, *et al.*, "Glm-4.5: Agentic, reasoning, and coding (arc) foundation models," *arXiv preprint arXiv:2508.06471*, 2025.
- [66] K. Team, A. Du, B. Gao, B. Xing, C. Jiang, C. Chen, C. Li, C. Xiao, C. Du, C. Liao, *et al.*, "Kimi k1.5: Scaling reinforcement learning with llms," *arXiv preprint arXiv:2501.12599*, 2025.
- [67] Y. Zhong, Z. Zhang, X. Song, H. Hu, C. Jin, B. Wu, N. Chen, Y. Chen, Y. Zhou, C. Wan, *et al.*, "Streamrl: Scalable, heterogeneous, and elastic rl for llms with disaggregated stream generation," *arXiv preprint arXiv:2504.15930*, 2025.

- [68] J. Li, Y. Jiang, Y. Zhu, C. Wang, and H. Xu, “Accelerating distributed {MoE} training and inference with lina,” in *USENIX ATC*, 2023.
- [69] C. Hwang, W. Cui, Y. Xiong, Z. Yang, Z. Liu, H. Hu, Z. Wang, R. Salas, J. Jose, P. Ram, *et al.*, “Tutel: Adaptive mixture-of-experts at scale,” *Proceedings of Machine Learning and Systems*, 2023.
- [70] X. Nie, P. Zhao, X. Miao, T. Zhao, and B. Cui, “Hetumoe: An efficient trillion-scale mixture-of-expert distributed training system,” *arXiv preprint arXiv:2203.14685*, 2022.
- [71] C. Jin, Z. Jiang, Z. Bai, Z. Zhong, J. Liu, X. Li, N. Zheng, X. Wang, C. Xie, Q. Huang, *et al.*, “Megascalmoe: Large-scale communication-efficient training of mixture-of-experts models in production,” *arXiv preprint arXiv:2505.11432*, 2025.
- [72] C. Chen, X. Li, Q. Zhu, J. Duan, P. Sun, X. Zhang, and C. Yang, “Centauri: Enabling efficient scheduling for communication-computation overlap in large model training via communication partitioning,” in *ACM ASPLOS*, 2024.
- [73] Z. Jiang, H. Lin, Y. Zhong, Q. Huang, Y. Chen, Z. Zhang, Y. Peng, X. Li, C. Xie, S. Nong, *et al.*, “{MegaScale}: Scaling large language model training to more than 10,000 {GPUs},” in *USENIX NSDI*, 2024.
- [74] S. Zhang, N. Zheng, H. Lin, Z. Jiang, W. Bao, C. Jiang, Q. Hou, W. Cui, S. Zheng, L.-W. Chang, *et al.*, “Comet: Fine-grained computation-communication overlapping for mixture-of-experts,” *Proceedings of Machine Learning and Systems*, 2025.
- [75] L.-W. Chang, W. Bao, Q. Hou, C. Jiang, N. Zheng, Y. Zhong, X. Zhang, Z. Song, C. Yao, Z. Jiang, *et al.*, “Flux: fast software-based communication overlap on gpus through kernel fusion,” *arXiv preprint arXiv:2406.06858*, 2024.
- [76] J. Liu, J. H. Wang, and Y. Jiang, “Janus: A unified distributed training framework for sparse mixture-of-experts models,” in *ACM SIGCOMM*, 2023.
- [77] G. Sheng, C. Zhang, Z. Ye, X. Wu, W. Zhang, R. Zhang, Y. Peng, H. Lin, and C. Wu, “Hybridflow: A flexible and efficient rlhf framework,” in *EuroSys*, 2025.
- [78] W. Gao, Y. Zhao, T. Wu, S. Xiong, W. Wang, D. An, L. Cao, D. Muhtar, Z. Liu, H. Zhao, *et al.*, “Rollart: Scaling agentic rl training via disaggregated infrastructure,” *arXiv preprint arXiv:2512.22560*, 2025.
- [79] Y. Leviathan, M. Kalman, and Y. Matias, “Fast inference from transformers via speculative decoding,” in *International Conference on Machine Learning (ICML)*, 2023.
- [80] C. Jin, Y. Zhong, Z. Zhang, Y. Jiang, and Y. Zhu, “Efficient rl for llms with dynamic and online speculative decoding,” in *Frontier AI Systems Workshop (FAISys)*, 2025.